

파인튜닝 및 Mobilint MLA100(ARIES) INT8 컴파일 — 절차 및 커맨드

YOLOv11s · Argoverse-HD | RTX 5090(FP32 학습) + Mobilint MLA100 / ARIES(INT8 컴파일)

EXP-FT-LOCAL 파이프라인의 자기완결형 기록. 모든 커맨드는 실제 실행한 것이며, 해시는 산출 파일의 SHA-256 앞자리입니다.

0. 목적과 핵심 설계 결정

목표: COCO 사전학습 YOLOv11s를 Argoverse-HD로 파인튜닝하고, Mobilint MLA100(ARIES) NPU용 INT8 모델로 컴파일한 뒤, COCO 모델과 파인튜닝 모델의 INT8 양자화 구조를 비교한다.

0.1 이 컴파일의 목적 — paired comparison (fine-tuning 효과 분리)

이 절차의 INT8 컴파일은 fine-tuning 효과를 비교하기 위한 paired compilation이다. 동일 dataset과 동일 recipe를 고정하고 모델 가중치만(COCO 사전학습 ↔ fine-tuned) 바꿔 한 쌍으로 컴파일한다. 다른 조건을 모두 묶어두므로, 두 INT8 모델의 차이는 곧 fine-tuning 효과로 해석할 수 있다(통계의 paired comparison과 같은 논리).

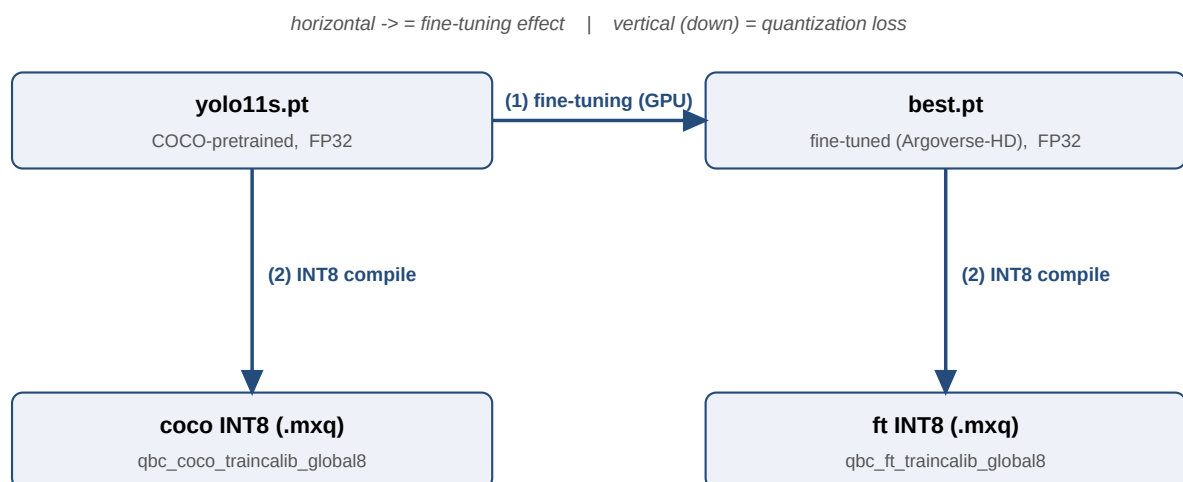
- 고정(동일): calibration data(train 200장, 같은 리스트), 평가 data(val 24 logs, 같은 protocol), compile recipe(qbcompiler, yolo_640 preset, global8, ONNX 640/opset13).
- 변화(비교 축): 모델 가중치만 — COCO-pretrained vs fine-tuned.

여기서 dataset은 비교 대상이 아니라 통제 변수(전제)다. fine-tuning 학습 자체는 Argoverse-HD train으로 수행하고(COCO 모델은 학습하지 않아 baseline 역할), 두 모델의 calibration·평가는 같은 data로 맞춘다. 정확히는 “동일 calibration·평가 data 위에서, Argoverse-HD train으로 fine-tuning한 효과가 INT8 quantization 구조에 어떻게 나타나는지 비교하는” 컴파일이다.

확인된 결과: fine-tuning으로 small-object 정확도를 올린 뒤에도 INT8의 small-biased quantization 구조가 유지됨(small 상대 손실 ≫ large) — 그 구조가 low-accuracy regime의 artifact가 아님을 보이는 것이 이 paired compilation의 목적이다.

0.2 비교 구조 — fine-tuning(학습) vs INT8 컴파일(양자화)은 별개 단계

두 단계는 전혀 다르다. ① fine-tuning은 GPU에서 모델 가중치(weight) 값을 학습으로 바꾸는 과정이며 타입은 계속 FP32다(NPU와 무관). ② INT8 컴파일은 학습이 끝난 모델을 NPU(ARIES)가 실행할 INT8 형식으로 quantization·패키징하는 과정으로, 가중치를 학습하는 게 아니라 형식을 바꾸는 것이다. 즉 fine-tuning은 NPU 컴파일을 의미하지 않는다.



그러므로 “다운로드한 float 모델”과 “컴파일한 int8 모델”은 가중치를 비교하는 두 모델이 아니라 한 모델의 두 형식(FP32 / INT8)이다. 측정은 아래 2×2 구조로 이뤄진다.

	FP32 (GPU)	INT8 (NPU, 컴파일 후)
--	------------	-------------------

COCO (원본, 학습 안 함)	yolo11s.pt	qbc_coco_traincalib_global8.mxq
fine-tuned (Argoverse-HD)	best.pt	qbc_ft_traincalib_global8.mxq

- 세로(↓, FP32→INT8) = 그 모델의 quantization 손실 (가중치 동일, 형식만 다름).
- 가로(→, COCO→fine-tuned) = fine-tuning 효과 (가중치가 다름).

paired comparison은 두 행의 quantization 손실 구조(세로 차이)를 나란히 비교한다.

왜 vendor blob이 아니라 로컬 쌍대 컴파일인가. 논문 본 실험의 NPU 모델은 vendor가 배포한 .mxq blob(HuggingFace mobilint/YOLO11s)이다. 검증 결과 로컬 컴파일러(qbcompiler 1.1.2)는 이 vendor blob을 허용오차 내로 재현하지 못했다(per-size sAP 차이 최대 0.008 > 0.003). 따라서 본 절차는 COCO와 파인튜닝 모델을 동일한 로컬 레시피로 모두 컴파일해, 레시피를 고정하고 가중치만 다르게 한 내부 통제 비교를 수행한다. 이 INT8 수치는 내부 COCO-vs-FT 비교용이며 vendor blob 기반 Table 1과 직접 비교하지 않는다.

1. 실행 환경 (가상환경 2개)

학습/추론과 컴파일은 분리된 파이썬 환경을 쓴다. Mobilint 컴파일러의 네이티브 모듈이 다른 PyTorch C++ ABI를 요구하기 때문이다.

(a) 학습/추론 `venv(.venv)`: torch 2.12.0+cu130, onnxruntime-gpu 1.20.1, ultralytics 8.4.56.

(b) 컴파일러 `venv(.venv_qbc)`: qbcompiler 1.1.2(+aries2). 네이티브 MMC 모듈이 C++11 ABI로 빌드돼 있어, cxx11-ABI이면서 CUDA(Blackwell sm_120)를 지원하는 PyTorch가 필요하다. torch 2.12(ABI 불일치) · torch 2.4(cxx11 아님) 모두 실패하고, torch 2.8.0+cu128(cxx11=True)이 작동 조합이다.

컴파일러 `venv` 구축 (최종 작동 순서):

확인: torch 2.8.0+cu128 (cxx11=True), onnxruntime 1.20.1, qbcompiler 1.1.2; qbcompiler.mmc import 성공, RTX 5090에서 CUDA 정상.

2. 단계 1 — 데이터셋 준비 (Argoverse-HD → YOLO 라벨)

Argoverse-HD train 어노테이션(COCO 형식, 8개 카테고리)을 Ultralytics YOLO 라벨로 변환한다. 라벨은 COCO-80 클래스 공간으로 기록한다(AHD id → COCO id 매핑 [0,1,2,3,5,7,9,11]). 이렇게 하면 파인튜닝 모델이 80-class head를 유지해 기존 평가 harness(COCO→AHD 매핑)를 그대로 재사용할 수 있다. 이미지는 symlink, 데이터셋 YAML을 생성한다.

생성된 데이터셋 YAML(핵심 필드):

3. 단계 2 — 파인튜닝 (RTX 5090, FP32 PyTorch)

Ultralytics 8.4.56 기본값, 고정 항목만 지정: 50 epoch, imgsz 640(=추론 해상도), seed 0, deterministic. 초기 가중치 = 공개 COCO 사전학습 `yolo11s.pt`. 세션과 무관하게 분리 실행.

옵티마이저는 Ultralytics auto(MuSGD, lr0=0.01로 결정). 결과: 50 epoch 완료, best checkpoint = epoch 1(Ultralytics 기본 기준 = val mAP 최고; mAP50-95가 epoch 1에서 0.236으로 정점 후 하락). cherry-pick 없이 best 그대로 사용: `ft_runs/ft_yolo11s/weights/best.pt`.

4. 단계 3 — ONNX export (컴파일러 입력)

컴파일러는 ONNX를 입력으로 받는다. COCO 사전학습 · 파인튜닝 가중치 모두 640×640, opset 13, 정적 shape, FP32로 export.

5. 단계 4 — INT8 양자화용 calibration 세트

INT8 양자화에는 대표 입력이 필요하다. Argoverse-HD train 프레임 200장(균등 샘플)을 고정해 두 컴파일에 공통으로 사용한다 — calibration을 COCO · FT 간 동일하게 두고 가중치만 다르게 한다.

6. 단계 5 — MLA100 / ARIES INT8(.mxq) 컴파일 (qbcompiler)

컴파일은 qbcompiler의 고수준 `mxq_compile()` API 사용(product = ARIES / MLA100). 레시피는 모델 간 고정:

- `config_preset = "yolo_640"`: YOLO 640×640 letterbox 전처리(pad 114), uint8 입력(정규화는 NPU가 수행), 3채널.
- `inference_scheme = "global8"`: 8코어 전체 모드, 단일 스트림(Table 1 방식) 평가용. N=4 멀티스트림

테스트용으로 "single" 모드(인스턴스당 1코어, 최대 8동시) 빌드도 생성.

- backend = "onnx", device = "gpu"(calibration은 호스트 GPU에서 수행).

각 컴파일은 약 30~33초 소요, ~10.7MB .mxq 산출(parse → 양자화/calibration → ARIES GlobalCluster 빌드 → export & 검증).

7. 단계 6 — 컴파일 후 검증

- 결정성/bit-identity: 각 .mxq를 20프레임 2회 실행 → box/score diff = 0.
- operating point: 단일 스트림 평가 threads=4, frame-skip ≈ 0.
- 컴파일 전 FP32 게이트: 파인튜닝 vs COCO offline mAP(small +0.045, medium +0.069)로 INT8 변환 전 정확도 상승을 확인.

8. 정확도 — fine-tuning 전/후 비교 (per-size sAP)

동일 24개 Argoverse-HD val log, 단일 스트림, threads=4(frame skip≈0), 3회 반복 평균. COCO(원본, 학습 안 함)와 fine-tuned 모델을 FP32(GPU) · INT8(NPU) 두 형식에서 측정한 streaming sAP.

sAP는 streaming(전달 시각 기준) 정확도, mAP는 offline(표준 COCO, 전달 지연 무관) 정확도다. 아래에 둘 다 제시한다.

모델 / 형식	all	small	medium	large
COCO — FP32 (GPU)	0.1957	0.0159	0.1839	0.4768
COCO — INT8 (NPU)	0.1874	0.0090	0.1535	0.4796
fine-tuned — FP32 (GPU)	0.2185	0.0485	0.2373	0.4559
fine-tuned — INT8 (NPU)	0.1920	0.0316	0.1976	0.4362

offline mAP @0.5:0.95 (위 sAP와 동일 harness · 24 val log · 3반복, per-log 평균):

모델 / 형식	offline mAP@0.5:0.95
COCO — FP32 (GPU)	0.2438
COCO — INT8 (NPU)	0.2311
fine-tuned — FP32 (GPU)	0.2700
fine-tuned — INT8 (NPU)	0.2339

(per-size offline mAP는 4셀 전부 저장돼 있지 않아 overall만 제시. FP32 per-size mAP는 아래 게이트 표 참고.)

참고 — 컴파일 전 FP32 게이트 (val 전수 pooled offline mAP, per-size; COCO vs fine-tuned). 위 per-log 집계와 방식이 달라 절대값이 다르다:

size	COCO (FP32)	fine-tuned (FP32)	Δ
all	0.1844	0.2131	+0.0287
small	0.0125	0.0576	+0.0451
medium	0.1734	0.2421	+0.0688
large	0.5029	0.4419	-0.0611

해석: fine-tuning으로 small·medium 정확도가 크게 상승(small은 ~3~4배), large는 소폭 하락. INT8(NPU)도 같은 경향을 따른다. 핵심은 fine-tuning으로 정확도를 올린 뒤에도 INT8의 small-biased quantization 구조(세로 차이: small 손실 ≫ large)가 유지된다는 점이다.

9. 산출물 (SHA-256 앞자리)

파일	역할	sha256[:16]
yolo11s.pt	COCO 사전학습 초기가중치(Ultralytics v8.2.100)	85a76fe86dd8afe3
ft_yolo11s/weights/best.pt	파인튜닝 FP32(epoch 1 best)	4c4b342a6d09de67

yololls.onnx	COCO ONNX(640, opset13)	5ffc807bb21ba5f5
best.onnx	파인튜닝 ONNX(640, opset13)	8e189e0252604e8f
qbc_coco_traincalib_global8.mxq	COCO INT8(global8)	f987aca54e64c363
qbc_ft_traincalib_global8.mxq	파인튜닝 INT8(global8)	ea091a9dc2485a79
qbc_ft_traincalib_single.mxq	파인튜닝 INT8(single)	c0520594743655e4

스크립트: ft_prep_dataset.py, ft_train_driver.sh, ft_dual_compile.py, ft_stretch_vlm.py (모두 accv_experiments/scripts/). 검출기 COCO 사전학습 가중치: Ultralytics GitHub assets 릴리스. NPU 컴파일러: qbcompiler 1.1.2(+aries2), Mobilint ARIES / MLA100.